L1 I.EEEA
Architecture des ordinateurs
TP 2 : Segmentation de la mémoire

Introduction

Déclaration des segments

Un code source assembleur doit contenir la déclaration des segments occupés par le programme au cours de son exécution. Par exemple, le code ci-dessous déclare le segment de code :

```
1 code segment
2 debut:
3
4 code ends
5     end debut
```

Les mots-clés `segment` et `ends` sont des directives d'assemblage [1] permettant de déclarer le segment.

Le mot `code` est le nom du segment (son étiquette). Par convention, on utilise l'étiquette `"code"` pour nommer le segment de code, mais c'est au choix du développeur.

Les lignes 2 et 5 quant à elles sont propres au segment de code et servent à délimiter les instructions du programme, qui seront à ajouter entre les lignes 2 et 4.

▶ Créez un nouveau fichier source assembleur `INTRO.ASM` et copiez-y le code ci-dessus. Ajoutez la déclaration du segment de données au dessus de celle du segment de code. Vous nommerez ce segment `data`.

Registre de segments

Pour exécuter un programme, le microprocesseur doit connaître les adresses des cases mémoires correspondant aux différents segments. Pour cela, il utilise les registres de segment :

— `CS` est le registre du segment de code

— `DS` est le registre du segment de données

Pendant l'exécution du programme, ces registres permettent d'identifier les cases mémoire des segments, et de spécifier les adresses logique de chacune de ces cases. Par exemple, si le registre `CS` contient la valeur `48EF`, la troisième case du segment de code est à l'adresse `CS:0002`, c'est-à-dire `48EF:0002`.

En assembleur, la directive `assume` permet de préciser à l'assembleur à quel segment déclaré dans le code correspond chacun des registres de segment. Par exemple :

```
assume CS:code

code segment
```

1. Les directives sont des commandes destinées à l'assembleur et non au processeur. Elles ne sont pas traduites en langage machine et ne sont pas inclues dans l'exécutable.

```
4       ...
5   code ends
```

Ici, on précise en ligne 1 que le contenu du registre `CS` correspond au début du segment nommé `code` dans le code source.

► Ajoutez cette directive à votre fichier source et complétez là pour en faire autant avec le segment de données. Ajoutez dans le segment de données, la déclaration d'une variable `VAL` au format octet (*byte*) contenant la valeur hexadécimale `42`.

Initialisation du registre `DS`

La valeur contenue dans le registre `CS` est automatiquement initialisée au chargement du programme. Cependant, ce n'est pas le cas pour le registre `DS`. Il faut impérativement insérer des instructions assembleur spécifiques au tout début du programme. Ces instructions sont :

```
1   mov AX, data
2   mov DS, AX
```

► Copiez ces instructions dans votre programme au tout début de votre segment de code (juste après l'étiquette `debut:`).

Pour pouvoir visualiser le contenu du segment de données avec Turbo Debugger, il faut exécuter ces deux instructions au préalable. Ensuite, le segment de données est visible dans la fenêtre `Dump` : menu `View` > `Dump`.

► Assemblez votre programme. S'il n'y a aucune erreur, lancez l'édition des liens et le déboguage du programme avec Turbo Debugger. Exécutez les deux premières instructions et ouvrez la fenêtre `Dump`. Vous devez pouvoir retrouver la valeur de `VAL` dans la première case mémoire.

Exercice 1 – Déclaration de variables dans le segment de données

1. Dans un dossier « TP2 », créez un fichier `EX1.ASM` et recopiez dans ce fichier le programme assembleur suivant :

```
1   assume CS:code
2
3   code segment
4   debut:
5       mov AX, data
6       mov DS, AX
7       mov AH, 9           ; ces 3 instructions permettent
8       lea DX, palindrome  ; d'afficher la chaine de caracteres
9       int 21h             ; palindrome dans le terminal
10      mov AX, 4C00h
11      int 21h
12  code ends
13      end debut
```

2. Complétez ce programme en déclarant le segment de données contenant :
   — une variable `A` contenant la valeur hexadécimale FAC sur deux octets
   — un tableau `premiers` contenant les valeurs 2, 3, 5, 7, 11, 13, 17, chacune de ces valeurs étant codée sur un octet.
   — une variable `B` contenant la valeur décimale 3141592653 sur 4 octets

— une chaîne de caractères `palindrome` contenant la phrase "Engage le jeu que je le gagne"

— une chaîne de caractères `erreur` contenant la phrase "–ERREUR : ce message ne doit pas s'afficher !"

— une variable `C` contenant la valeur décimale −99 sur un octet

— une variable `vitesse_lumiere` contenant la valeur décimale 300000

— un tableau `temperatures` contenant les valeurs −200, −100, 0, 100 et 200, sur le moins d'octets possibles

3. Lancez l'assemblage, l'édition des liens et exécutez le programme dans le terminal en saisissant `EX1.EXE` dans le terminal. Quel résultat obtenez vous ?
4. Déboguez votre programme avec Turbo Debugger et exécutez les deux premières instructions. Quel est le résultat produit ?
5. Affichez le contenu du segment de données dans Turbo Debugger. Combien d'octets occupe-t-il ?
6. Représentez les 20 premiers octets du segment de données dans un tableau, en indiquant leurs adresses logiques et à quelles variables ils appartiennent.
7. Quelle est la représentation hexadécimale de la valeur de `B` ? de `C` ? de `vitesse_lumiere` ?

Exercice 2 – Décompilation

On suppose qu'on a correctement écrit, assemblé et lié le code source d'un programme assembleur. Le déboguage et l'exécution des deux premières instructions du programme avec Turbo Debugger ont permis d'obtenir la fenêtre de la figure 1.

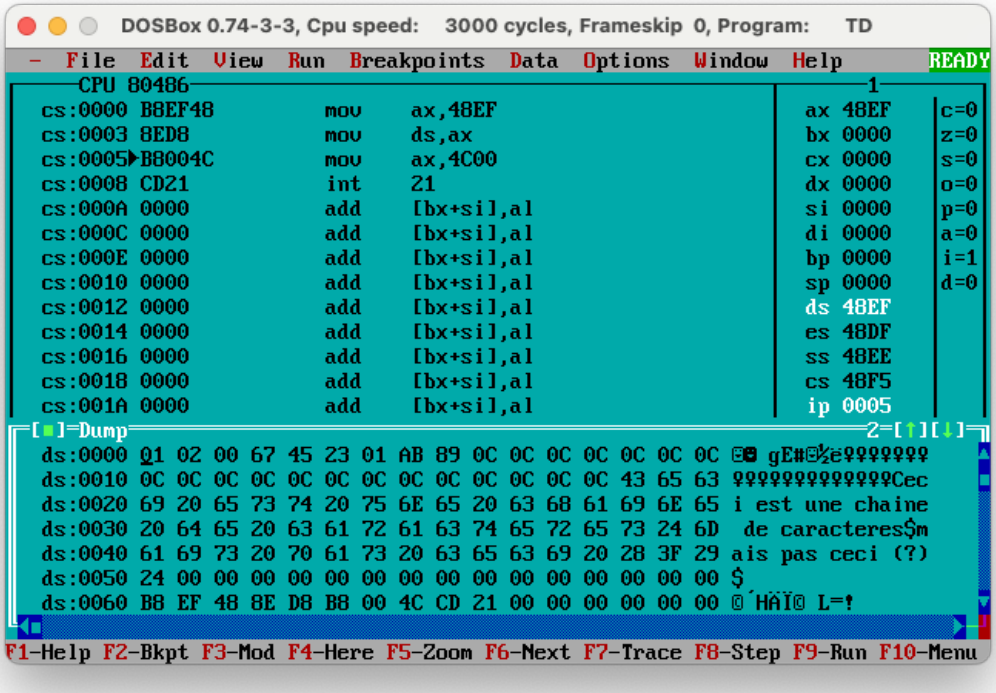

FIGURE 1 –

L'objectif de cet exercice est de reconstituer le fichier source du programme à partir des informations observées sur la figure 1.

1. Créez un fichier `EX2.ASM` et déclarez-y un segment de données et un segment de code vide.
2. En examinant la fenêtre `Dump` (cadre en bas de la figure 1) proposez des déclarations de variables qui permettraient d'aboutir à ce contenu de la mémoire. Pour vous guider :
   - Les trois premiers octets correspondent à trois variables différentes
   - Les trois formats de variables sont utilisés
   - L'interprétation ASCII des octets donne des indications
   - Comme souvent, la fin du segment de données est suivi de près par le début du segment de code.
3. Complétez le fichier en ajoutant les instructions du programme que vous visualisez sur la figure 1
4. Assemblez, lié et déboguez ce programme pour vérifier que vous obtenez le même résultat qu'en figure 1